

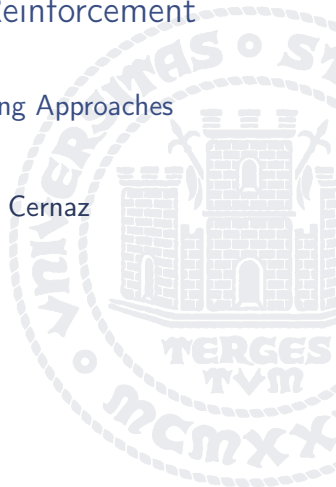
Learning to Play Briscola with Reinforcement Learning

A Comparison of Reinforcement Learning Approaches

Alessandro Longato Federico Cernaz

University of Trieste
Reinforcement Learning Course

30th June, 2026



Project Motivation

Briscola is a popular Italian card game that falls into the category of Partially Observable Stochastic Games (POSG).

- ▶ Partially Observable: the opponent's cards are hidden.
- ▶ Stochastic: the cards are dealt at random.

Moreover, it requires long-term strategic planning:

- ▶ Future rewards depend on current decisions.
- ▶ Strong cards may be conserved or sacrificed strategically.
- ▶ Non-trivial trade-off between short-term and long-term gains

Objective: To compare different algorithms to train RL agents capable of learning effective Briscola strategies.

Environment Properties

- ▶ **Extensive State Space:** The number of possible game situations is enormous due to card permutations, making standard tabular methods intractable without abstraction.
- ▶ **Zero-Sum Game:** The game is strictly competitive. The rewards sum to zero ($R_1 + R_2 = 0$), meaning any gain for Player 1 is a direct loss for Player 2.
- ▶ **Episodic with Finite Horizon:** The game ends after 40 turns. Because the episode is finite and the goal is simply to maximize total points by the end of the game, no discount factor is needed ($\gamma = 1$). A point earned on turn 39 is just as valuable as a point earned on turn 1.

Action Space

Each action corresponds to playing a card from the current hand.

Example:

- ▶ Hand = Ace, Three, King
- ▶ Legal actions = Play Ace, Play Three, Play King

The environment automatically restricts actions to valid cards, which are defined as the cards in the active player's hand.

Reward Function

Reward shaping was introduced to provide denser training feedback.

Immediate reward:

$$R_t = 0.2 \times \text{Points obtained from the trick}$$

Terminal reward:

- +50 for winning the game
- −50 for losing the game

This combines local feedback with the final game outcome.

Why Reward Shaping?

Using only terminal rewards would provide sparse feedback.

Advantages of reward shaping:

- ▶ Faster learning.
- ▶ More informative training signals.
- ▶ Easier credit assignment.
- ▶ Improved convergence stability.



Independent Learning

- ▶ The first class of explored algorithms is Independent Learning algorithms, which reduce the multi-agent setting to a single-agent one.
- ▶ Agents do not observe or use information about other agents, and the effects of other agents' actions are simply part of the environmental dynamics from the perspective of each learning agent.

Independent Q-Learning

Independent Tabular Q-Learning learns a value $Q(s, a)$ for every observable state-action pair, updating estimates through repeated interactions with the environment.

- ▶ **State:** represented as a symbolic string (player, hand, briscola, table card).
- ▶ **Action:** selecting a card from those in hand.
- ▶ **Policy:** ϵ -greedy over the learned Q-values.

This method directly stores and updates values for each visited state.

Independent Q-Learning

Algorithm 1 Independent Q-learning (IQL) for stochastic games

- 1: Initialize: $Q_i(s, a_i) = 0$ for all $s \in S, a_i \in A_i$
 - 2: Repeat for every episode:
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Observe current state s^t
 - 5: With probability ϵ : choose random action $a_i^t \in A_i$
 - 6: Otherwise: choose action $a_i^t \in \arg \max_{a_i} Q_i(s^t, a_i)$
 - 7: (meanwhile, other agents $j \neq i$ choose their actions a_j^t)
 - 8: Observe own reward r_i^t and next state s^{t+1}
 - 9: $Q_i(s^t, a_i^t) \leftarrow Q_i(s^t, a_i^t) + \alpha[r_i^t + \gamma \max_{a'_i} Q_i(s^{t+1}, a'_i) - Q_i(s^t, a_i^t)]$
-

State Representation (IQL)

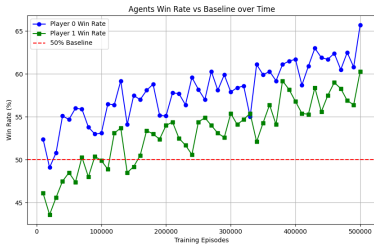
The tabular agent uses a symbolic information-state string that uniquely identifies the observable configuration.

Component	Description
Player ID	Active player (0 or 1)
Hand	Sorted list of visible cards
Briscola suit	Briscola suit index (0–3)
Table card	Card currently on the table

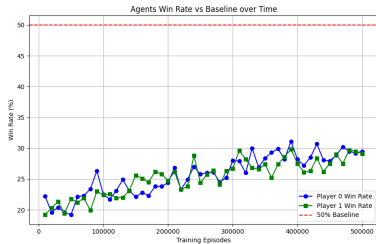
Example

P0 | Hand:7,14,33 | Briscola:1 | Table:22

IQL: Learning Curves



Learning Curve against Random



Learning Curve against Heuristic

IQL: Strengths and Limitations

Strengths

- ▶ **Simple and interpretable:** Q-values can be inspected directly.
- ▶ **Exact updates:** No function approximation error.

Limitations

- ▶ **State explosion:** The number of possible states grows combinatorially.
- ▶ **Memory heavy:** The Q-table grows with every new state encountered, making it impractical for large state spaces.
- ▶ **Poor generalization:** Unseen states have no Q-value.
- ▶ **Non-stationarity:** Since the policies of other agents keep changing, learning may be unstable.

Moving to Deep Learning

To make up for the shortcomings of Tabular Learning, we moved towards Deep Learning.

Deep Reinforcement Learning offers several benefits:

- ▶ **Function Approximation:** Neural networks act as powerful function approximators that map states to action values.
- ▶ **Generalization:** They can generalize value estimation for states that are similar, but not identical, to previously encountered states.

The Moving Target Problem

Using function approximation with **off-policy learning** and **bootstrapped targets** creates the **deadly triad**, leading to severe instability and diverging value estimates.

- ▶ Because neural networks generalize, updating the value for a single state may accidentally change the value estimates of all other states.
- ▶ The target $y^t = r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \theta)$ uses the same parameters θ that are being updated.
- ▶ This causes the bootstrapped target to change rapidly and unpredictably, making optimization highly unstable.

Target Networks

To stop the target from rapidly changing, an additional **target network** with parameters $\bar{\theta}$ is introduced.

- ▶ The target network has the exact same architecture as the main value function network.
- ▶ Target values are computed using this frozen network:
$$y^t = r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \bar{\theta})$$
- ▶ Instead of using gradient descent, the target parameters $\bar{\theta}$ are periodically updated by copying the main network's parameters ($\bar{\theta} \leftarrow \theta$).
- ▶ **Benefit:** Decouples the target computation from the main network, providing stable targets for a fixed number of steps and mitigating divergence.

Correlated Experiences

Machine learning optimization assumes that data is **independent and identically distributed** (i.i.d.). RL severely violates this:

- ▶ **High Correlation:** Sequential states s^t and s^{t+1} are highly dependent on one another.
- ▶ **Changing Distributions:** The data distribution relies heavily on the agent's current policy, which changes continuously during training.

Because consecutive samples are highly correlated, the neural network overfits to the most recent experiences. For example, if an agent spends multiple episodes moving left, it will forget how to move right.

Experience Replay Buffer

Instead of learning from sequential experiences as they occur, the agent stores transition tuples (s^t, a^t, r^t, s^{t+1}) into a dataset called a **replay buffer** $\mathcal{D}$.

- ▶ For training, the algorithm samples a **random mini-batch** $\mathcal{B} \sim \mathcal{U}(\mathcal{D})$.
- ▶ **Stability:** Random sampling breaks the temporal correlation of data, making it closer to i.i.d. and preventing catastrophic forgetting.
- ▶ **Sample Efficiency:** Rare or expensive experiences can be reused multiple times to update the network.
- ▶ **Lower Variance:** Computing losses over a batch provides a more stable gradient.

IDQN Algorithm

Algorithm 2 Independent Deep Q-Networks

- 1: Initialize n value networks with random parameters $(\theta_1, \dots, \theta_n)$
- 2: Initialize n target networks with parameters $(\bar{\theta}_1 = \theta_1, \dots, \bar{\theta}_n = \theta_n)$
- 3: Initialize a replay buffer for each agent $D_1, \dots, D_n$
- 4: **for** time step $t = 0, 1, 2, \dots$ **do**
- 5: Collect current observations $o_1^t, \dots, o_n^t$
- 6: **for** agent $i = 1, \dots, n$ **do**
- 7: With probability ϵ : choose random action a_i^t
- 8: Otherwise: choose $a_i^t \in \arg \max_{a_i} Q(h_i^t, a_i; \theta_i)$
- 9: Apply joint actions $(a_1^t, \dots, a_n^t)$; collect rewards $r_1^t, \dots, r_n^t$ and next obs
- 10: **for** agent $i = 1, \dots, n$ **do**
- 11: Store transition $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$ in replay buffer D_i
- 12: Sample mini-batch of B transitions $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$ from D_i
- 13: **if** s^{k+1} is terminal **then**
- 14: Target $y_i^k \leftarrow r_i^k$
- 15: **else**
- 16: Target $y_i^k \leftarrow r_i^k + \gamma \max_{a_i'} Q(h_i^{k+1}, a_i'; \bar{\theta}_i)$
- 17: Loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^B (y_i^k - Q(h_i^k, a_i^k; \theta_i))^2$
- 18: Update θ_i by minimizing $\mathcal{L}(\theta_i)$
- 19: Periodically update target network $\bar{\theta}_i \leftarrow \theta_i$

State Representation (DQN)

The Deep Q-Learning agent receives a fixed-size information-state vector containing only information observable to the acting player.

Feature	Size
Player hand (multi-hot)	40
Briscola suit (one-hot)	4
Current table card (one-hot)	41
Played cards (multi-hot)	40
Own score (normalized)	1
Opponent score (normalized)	1
Remaining deck size	1
Leading player flag	1
Total	129

IDQN: Learning Curves



Learning Curve against Random



Learning Curve against Heuristic

Strengths & Weaknesses of IDQN

Strengths:

- ▶ **Generalization:** Successfully scales to enormous state spaces.
- ▶ **Stable off-policy learning:** Effectively overcomes the deadly triad using target networks.
- ▶ **High Sample Efficiency:** Replay buffers allow past data to be safely reused.

Weaknesses

- ▶ DQN assumes a stationary environment. From the perspective of any single agent, the environment's dynamics are constantly changing, destabilizing the replay buffer and breaking convergence.

Moving to Self Play

By having an agent play against copies of itself and sharing the neural network weights across agents, sample efficiency is drastically improved and the opponent always matches the agent's skill level.

$$\theta_{\text{shared}} = \theta_1 = \theta_2 = \dots = \theta_n$$

However, Self-Play DQN suffers from catastrophic forgetting in a strategic sense. It overfits to its current self and forgets how to beat older versions. This leads to an endless cycle, preventing the algorithm from converging to a robust, unbeatable strategy.

Self Play: Learning Curves



Learning Curve against Random



Learning Curve against Heuristic

Moving to PSRO

- ▶ DQN only maximizes reward against the current opponent distribution, rather than finding a balanced Nash Equilibrium.
- ▶ Policy Space Response Oracles solves these cyclic dynamics by maintaining a **population** of diverse historical policies. By using an RL agent (like DQN) as an "oracle" to compute best responses to a meta-game of past policies, PSRO guarantees robust game-theoretic convergence.

PSRO Algorithm

Algorithm 3 Policy Space Response Oracle

- 1: Initialize populations Π_i^1 for all $i \in I$ (e.g., random policies)
 - 2: **for** each generation $k = 1, 2, 3, \dots$ **do**
 - 3: Construct meta-game M^k from current populations $\{\Pi_i^k\}_{i \in I}$
 - 4: Use meta-solver on M^k to obtain distributions $\{\delta_i^k\}_{i \in I}$
 - 5: **for** each agent $i \in I$ **do**
 - 6: **for** each episode $e = 1, 2, 3, \dots$ **do**
 - 7: Sample policies for other agents $\pi_{-i} \sim \delta_{-i}^k$
 - 8: Use single-agent RL to train π'_i wrt. π_{-i} in underlying game G
 - 9: Grow population $\Pi_i^{k+1} \leftarrow \Pi_i^k \cup \{\pi_i\}$
-

PSRO: Strengths

- ▶ **Guaranteed Convergence:** PSRO mathematically guarantees convergence toward a Nash Equilibrium.
- ▶ **Unexploitable Mixed Strategies:** By calculating a probability distribution over the meta-game matrix, the agent plays a randomized mixed strategy, making it impossible for the opponent to predict its behavior.
- ▶ **No Forgetting:** Because previous policies are frozen and kept in a population, the agent never suffers from catastrophic forgetting.

PSRO: Limitations

- ▶ **High Computational Cost:** Every single iteration requires training a new neural network from scratch.
- ▶ **The "Specialist" Problem:** An Oracle trained in iteration N is highly specialized to beat iterations 1 through $N - 1$. If deployed as a single agent rather than part of a mixed strategy, it will overfit and perform poorly against unknown opponents.
- ▶ **Slow Meta-Game Expansion:** Constructing a truly robust meta-game requires dozens of iterations. With limited compute, small meta-games (e.g., 5 iterations) lack the strategic diversity needed to dominate.

The Failure of Deep CFR

- ▶ To find an exact Nash Equilibrium, we attempted to train an agent using Deep Counterfactual Regret Minimization (Deep CFR), the algorithm that successfully solved No-Limit Texas Hold'em.
- ▶ Deep CFR uses external sampling. It samples chance nodes and opponent actions, but recursively branches to explore every possible action at the acting player's decision nodes.
- ▶ However, Briscola possesses an unmanageable branching factor for this approach. Since an agent holds up to 3 cards per turn and a game lasts 20 turns per player, a single game traversal attempts to visit about 3^{20} recursive nodes.

The Failure of Deep CFR

- ▶ Thus, the algorithm suffered from a tree traversal explosion. Training froze indefinitely as the CPU attempted to calculate an absurd number of hypothetical game states per iteration.
- ▶ Algorithms relying on recursive tree traversal are computationally intractable to train on a consumer CPU, requiring an enormous amount of computing power.

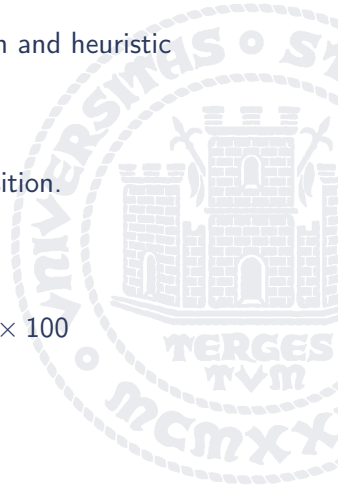
Evaluation Protocol

The trained agent is evaluated against random and heuristic opponents.

Evaluation procedure:

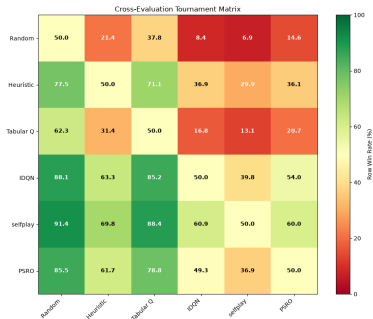
- ▶ 1000 independent games per starting position.
- ▶ Exploration disabled.
- ▶ Performance measured using win rate.

$$\text{Win Rate} = \frac{\text{Number of wins}}{\text{Total games}} \times 100$$



Tournament Results

To determine the true strength of our algorithms, we pitted all trained agents against each other in a round-robin tournament with 1000 games per matchup.



Tournament Matrix Result

Discussion

- ▶ **Curse of dimensionality:** Function approximation and deep learning are absolutely necessary for complex card games. Tabular methods fail entirely without strong state abstraction.
- ▶ **Parameter sharing:** In games with symmetric rules but asymmetric turn orders, sharing neural network weights allows agents to learn robust strategies much faster.
- ▶ **Meta-Games:** While PSRO is theoretically sound, evaluating it as a mixed strategy across only 5 iterations limits its strategic diversity. To truly outperform standard self play, PSRO requires massive computational scale .

Conclusions

- ▶ Successfully implemented a fully functional, multi-agent RL environment for Briscola using the OpenSpiel framework.
- ▶ Designed a 129-dimension information state tensor alongside a robust reward shaping mechanism.
- ▶ Successfully trained, evaluated, and compared four distinct RL architectures: Independent Tabular Q-Learning, Independent DQN, Self-Play DQN, and PSRO.
- ▶ Demonstrated that deep RL agents can successfully surpass hard-coded heuristic strategies and learn complex, long-term planning in partially observable stochastic games.

Thank You
Any questions?

